

Разделение проекта на двух участников

Илья и Павел

1 Контекст проекта

1.1 Точное условие проектного задания

Ниже приводится условие проекта в формулировке, принятой за исходную:

Проектное задание (оптимизация)

Рассмотрите статью (приложена)

Ключевой запрос в поисковую систему: `loss landscape visualisation`

Локализуите вычисление функции потерь при обучении модели глубокого обучения: лучше адаптировать вариант с GitHub'a из приложенной статьи либо аналогичной

Нужна незначительная доработка

Пример как выглядит визуализация в проекции на 2 координаты приложен

После локализации предложите приближение функции потерь некоторой выпуклой гладкой функцией (например, квадратичной положительно определенной); вариант: можно рассмотреть матрицу Гессе функции потерь

Создайте процедуру вычисления нормы разности функции и ее приближения

1.2 Как понимается это условие

Из условия следует, что проект состоит из нескольких обязательных содержательных частей:

1. изучить статью и понять её основную идею;
2. локализовать вычисление функции потерь в окрестности обученного решения нейросети;
3. построить визуализацию `loss landscape`, прежде всего в двумерной проекции;
4. предложить гладкое выпуклое приближение полученного локального сечения;
5. ввести способ количественно измерять ошибку между реальной поверхностью и её приближением.

Следовательно, главная логика проекта должна идти по цепочке

`loss landscape` $\longrightarrow$ локальное 2D-сечение $\longrightarrow$ выпуклая гладкая аппроксимация $\longrightarrow$ норма разности

Именно эта цепочка и была принята как основной содержательный центр будущей работы.

1.3 Кратко о статье

В качестве базовой статьи рассматривается работа *Visualizing the Loss Landscape of Neural Nets*. Её основная идея заключается в том, что функцию потерь нейросети нельзя визуализировать во всём пространстве параметров напрямую, поскольку это пространство очень большой размерности. Поэтому исследуются **сечения** функции потерь в выбранных направлениях около некоторого обученного решения.

Если обозначить параметры обученной модели через θ , а выбранные направления в пространстве параметров через d_1, d_2 , то рассматриваемая поверхность имеет вид

$$f(\alpha, \beta) = L(\theta^{+\alpha d_1 + \beta d_2}).$$

Именно такие сечения позволяют увидеть локальную геометрию функции потерь:

- насколько минимум выглядит “острым” или “плоским”;
- насколько поверхность похожа на чашеобразную;
- есть ли выраженная анизотропия;
- есть ли визуальные признаки сильной невыпуклости.

В статье отдельно подчёркивается, что простое сравнение поверхностей в пространстве параметров может быть обманчивым из-за масштабных симметрий и особенностей параметризации сети. Поэтому большое значение имеет корректный выбор и нормировка направлений.

Кроме того, статья показывает, что архитектура сети влияет на геометрию loss landscape: residual-связи, ширина сети и другие архитектурные решения могут делать поверхность более регулярной или, наоборот, более хаотичной. Однако для нашего проекта это оказалось не главным сюжетом, а скорее возможным дополнительным экспериментом.

1.4 Ключевой вывод из статьи для нашего проекта

Главный вывод, который был вынесен из статьи применительно к нашему проекту, можно сформулировать так:

Наиболее важным для проектного задания является не само сравнение архитектур, а идея локального двумерного сечения функции потерь и возможность аппроксимировать это сечение простой выпуклой гладкой функцией.

Именно поэтому основной фокус проекта был смещён с темы “какая сеть лучше” на тему

насколько хорошо local loss landscape допускает простую выпуклую surrogate-модель.

1.5 Как изначально обсуждалась структура проекта

На первом этапе обсуждения рассматривался вариант сделать акцент на сравнении двух близких архитектур:

- residual-сети;

- той же сети без residual shortcut connections.

Этот вариант имел смысл по следующим причинам:

1. он хорошо соотносится со статьёй;
2. он даёт визуально интересное сравнение;
3. его удобно объяснять на защите.

Однако затем было замечено, что при таком акценте проект начинает выглядеть скорее как работа “про архитектуры нейросетей”, а не как проект по оптимизации и локальной аппроксимации функции потерь.

Поэтому было принято более точное решение: сравнение архитектур оставить, но только как **дополнительную проверку**, а основным содержанием сделать **сравнение аппроксимаций**.

1.6 Итоговая концепция проекта после обсуждения

После всех обсуждений была зафиксирована следующая центральная идея проекта:

Исследовать локальное двумерное сечение функции потерь обученной нейросети и сравнить несколько способов его выпуклой гладкой аппроксимации по качеству приближения.

В такой постановке:

- одна обученная модель даёт исходный local loss landscape;
- этот landscape вычисляется на сетке в координатах (α, β) ;
- затем к нему подгоняются несколько выпуклых гладких функций;
- после этого сравнивается, какая из аппроксимаций лучше описывает исходную поверхность.

1.7 Какие именно аппроксимации были выбраны

В результате обсуждения было решено, что наиболее разумно сравнивать следующие варианты:

1. Полная выпуклая квадратичная модель

$$q_1(\alpha, \beta) = c + u\alpha + v\beta + \frac{1}{2}(a\alpha^2 + 2b\alpha\beta + d\beta^2),$$

где соответствующая матрица квадратичной формы положительно определена.

Эта модель была выбрана как основная, потому что она:

- достаточно простая;
- хорошо интерпретируема;
- умеет учитывать смешанный член $\alpha\beta$;
- естественно описывает локальную чашеобразную поверхность.

2. Диагональная выпуклая квадратичная модель

$$q_2(\alpha, \beta) = c + u\alpha + v\beta + \frac{1}{2}(a\alpha^2 + d\beta^2).$$

Эта модель была выбрана как baseline, чтобы проверить, насколько важен смешанный член и насколько поверхность действительно требует более общей квадратичной формы.

3. Hessian-based квадратичная модель

Дополнительно был рассмотрен вариант строить локальную квадратичную модель через вторые производные в выбранной двумерной плоскости. Этот вариант признан полезным, но не обязательным. Он был оставлен как **усиление проекта** или бонусная часть.

1.8 Какие метрики сравнения были признаны основными

Было решено, что сравнение аппроксимаций должно быть количественным, а не только визуальным.

Поэтому в качестве основных метрик были выбраны:

- RMSE как средняя типичная ошибка приближения;
- L_∞ как наихудшее локальное отклонение;
- Relative RMSE как дополнительная относительная метрика.

То есть проект должен отвечать не просто на вопрос

“какая картинка выглядит лучше”,

а на вопрос

“какая выпуклая аппроксимация даёт меньшую ошибку по строгим количественным критериям”.

1.9 Почему важен радиус локальной области

В обсуждении отдельно был выделен ещё один важный параметр — радиус рассматриваемой окрестности.

Если поверхность рассматривается в области

$$(\alpha, \beta) \in [-r, r] \times [-r, r],$$

то при маленьком r квадратичная модель естественно должна работать лучше, а при увеличении r начинают сказываться эффекты более высокого порядка.

Поэтому было решено включить в проект отдельный анализ:

как зависит качество выпуклой аппроксимации от радиуса локальной области.

Этот пункт был признан одним из самых сильных математических аспектов проекта.

1.10 Почему сравнение двух архитектур всё же оставлено

Хотя главным было решено сделать сравнение аппроксимаций, обсуждение показало, что полностью отказываться от второй архитектуры не стоит.

Сравнение двух близких моделей имеет смысл как дополнительный эксперимент:

- чтобы проверить, не являются ли выводы случайными для одной конкретной сети;
- чтобы посмотреть, как те же аппроксимации ведут себя на более и менее регулярных поверхностях;
- чтобы усилить экспериментальную часть проекта.

Но было отдельно зафиксировано, что этот архитектурный эксперимент не должен стать центром работы. Его роль — **подтверждающая, а не основная**.

1.11 Как в итоге была выбрана финальная структура проекта

После обсуждения была принята следующая структура:

Основная часть

1. обучить одну компактную ResNet-like модель на CIFAR-10;
2. построить 1D и 2D local loss landscape;
3. сравнить несколько выпуклых гладких аппроксимаций;
4. посчитать нормы ошибки;
5. исследовать влияние радиуса.

Дополнительная часть

1. взять вторую архитектуру;
2. повторить на ней сокращённый основной анализ;
3. проверить устойчивость выводов.

1.12 Как была распределена работа между участниками

После того как проект был переформулирован, стало ясно, что его удобно делить на двух человек не “поровну по сложности”, а по типу работы.

- **Илья** берёт на себя всю инженерно-черновую и организационно-вычислительную часть:
 - структура проекта;
 - запуск кода;
 - обучение моделей;
 - прогон вычислений;
 - сбор артефактов;

— графики, таблицы, оформление.

• **Павел** отвечает за математическое ядро:

- постановка задачи;
- выбор аппроксимаций;
- выбор норм;
- интерпретация результатов;
- формулировка выводов;
- содержательная часть защиты.

Такое разделение было признано естественным и продуктивным, потому что проект реально состоит из двух разных по характеру частей: инженерной и аналитико-математической.

1.13 Финальный смысл всего контекста

Итог всей предварительной работы можно сформулировать так:

Проект строится как исследование локальной геометрии функции потерь обученной нейросети через двумерные сечения и последующее сравнение нескольких выпуклых гладких аппроксимаций этой поверхности. Архитектурное сравнение используется только как дополнительная проверка устойчивости результатов.

То есть в окончательной версии проекта центральным становится именно вопрос:

какая аппроксимация лучше описывает `local loss landscape` и в какой области это приближение остаётся качественным.

1.14 Переход к следующей части документа

После этого контекстного блока в документе логично переходить к:

1. подробному плану проекта;
2. распределению задач между Ильёй и Павлом;
3. структуре отчёта;
4. этапам реализации.

2 Общий принцип разделения

Проект делится не по принципу “поровну по сложности”, а по принципу **разных типов работы**.

- **Илья** отвечает в первую очередь за черновую, организационную, кодово-запусковую и оформительскую часть.
- **Павел** отвечает в первую очередь за математическое ядро проекта, постановку аппроксимаций, анализ результатов и содержательную логику.

Такое разделение является разумным, потому что сам проект состоит из двух сильно разных частей:

- 1) инженерно-вычислительная часть;
- 2) математико-исследовательская часть.

3 Роли участников

3.1 Илья

Илья отвечает за:

- создание и поддержку структуры проекта;
- работу с окружением, зависимостями и запуском;
- обучение моделей;
- прогон скриптов и получение артефактов;
- сохранение результатов;
- построение графиков и таблиц;
- сборку отчёта в единый документ;
- техническую часть презентации и итогового оформления.

Илья — это **инженер-реализатор и сборщик пайплайна**.

3.2 Павел

Павел отвечает за:

- математическую постановку проекта;
- формулировку основной гипотезы и дополнительной гипотезы;
- выбор и описание аппроксимирующих моделей;
- выбор метрик ошибки и их интерпретацию;
- анализ радиуса локальности;

- интерпретацию результатов;
- формулировку выводов;
- смысловую часть защиты.

Павел — это **аналитик-исследователь и автор математического ядра**.

4 Как делится наш проект в новой постановке

Главный фокус проекта:

сравнение выпуклых аппроксимаций local loss landscape, а не сравнение архитектур как таковое.

Архитектурный эксперимент со второй моделью остаётся, но только как дополнительный подтверждающий блок.

4.1 Основной сюжет проекта

- 1) обучить одну компактную ResNet-like модель на CIFAR-10;
- 2) построить 1D и 2D local loss landscape;
- 3) сравнить несколько аппроксимаций:
 - полную выпуклую квадратичную;
 - диагональную выпуклую квадратичную;
 - Hessian-based как бонус.
- 4) посчитать RMSE, L_∞ , Relative RMSE;
- 5) проверить влияние радиуса локальной области.

4.2 Дополнительный сюжет проекта

- 1) взять вторую архитектуру;
- 2) повторить на ней укороченный основной анализ;
- 3) проверить, сохраняются ли выводы про качество аппроксимаций.

5 Разделение ответственности по блокам

5.1 Блок 1. Структура проекта и кодовая база

Илья

- создаёт структуру папок проекта;
- поднимает окружение и зависимости;
- создаёт skeleton файлов;
- настраивает конфиги, пути, сохранение результатов;
- собирает CLI-скрипты запуска.

Павел

- проверяет, что структура кода соответствует логике исследования;
- определяет, какие модули обязательны:
 - `surface.py`,
 - `quadratic_fit.py`,
 - `metrics.py`,
 - `plotting.py`,
 - `hessian_2d.py`.

Итог

Илья делает руками, Павел утверждает исследовательскую структуру.

5.2 Блок 2. Обучение модели и получение loss landscape

Илья

- запускает обучение основной модели;
- следит за checkpoint-файлами;
- прогоняет 1D и 2D скрипты;
- сохраняет поверхности, графики, таблицы;
- повторяет те же действия для второй модели при необходимости.

Павел

- определяет, какую основную модель использовать;
- выбирает вторую модель для дополнительного эксперимента;
- задаёт диапазоны для α и β ;
- задаёт размер сетки;
- решает, какие именно графики обязательны.

Итог

Илья добывает вычислительные результаты, Павел определяет, какие результаты нужны содержательно.

5.3 Блок 3. Математическая постановка

Илья

- переносит формулы в LaTeX;
- оформляет математические блоки;
- помогает связать формулы с кодом.

Павел

- задаёт основную функцию

$$f(\alpha, \beta) = L(\theta^{+\alpha d_1 + \beta d_2});$$

- выбирает семейство аппроксимаций;
- формулирует условия выпуклости;
- определяет, какие нормы считать;
- задаёт исследовательские гипотезы.

Итог

Павел полностью отвечает за математическое ядро, Илья помогает его материализовать в документе и коде.

5.4 Блок 4. Сравнение аппроксимаций

Илья

- прогоняет fit-скрипты;
- собирает результаты по всем аппроксимациям;
- формирует таблицы ошибок;
- строит графики ошибок и heatmap'ы.

Павел

- определяет, какие аппроксимации являются обязательными;
- решает, что считать baseline;
- интерпретирует, почему одна аппроксимация лучше другой;
- формулирует смысл сравнений.

Итог

Илья собирает и оформляет результаты, Павел делает их осмысленный анализ.

5.5 Блок 5. Анализ радиуса локальной области

Илья

- запускает расчёты для нескольких радиусов;
- сохраняет таблицы и графики зависимости ошибки от радиуса.

Павел

- выбирает набор радиусов;
- интерпретирует зависимость качества аппроксимации от радиуса;
- формулирует выводы о локальности quadratic surrogate.

Итог

Илья делает серию запусков, Павел отвечает за смысл результатов.

5.6 Блок 6. Дополнительный эксперимент со второй архитектурой

Илья

- обучает или запускает вторую модель;
- повторяет базовый pipeline;
- собирает результаты и оформляет сравнительные таблицы.

Павел

- решает, нужна ли вторая архитектура;
- определяет, какие именно аппроксимации повторять на второй модели;
- анализирует, насколько устойчивы основные выводы.

Итог

Вторая архитектура — это дополнительная проверка, и её технически тянет Илья, а концептуально — Павел.

6 Разделение написания отчёта

6.1 Что пишет Илья

Илья отвечает за следующие части текста:

- обзор статьи и связанного репозитория;
- описание структуры проекта и реализации;
- технические детали pipeline;
- оформление рисунков, таблиц и подписей;
- перенос текста и результатов в `.tex`;
- сборку финального документа.

6.2 Что пишет Павел

Павел отвечает за следующие части текста:

- постановка задачи;
- математическая часть;
- описание и обоснование аппроксимаций;
- метрики и их интерпретация;
- результаты и обсуждение;
- выводы;
- смысловая логика защиты.

6.3 Итоговая модель

Павел пишет смысл, Илья собирает, оформляет и доводит.

7 Разделение защиты

7.1 Что должен рассказывать Илья

Илья должен уверенно рассказывать:

- как устроен pipeline проекта;
- как была организована реализация;
- как обучались модели;
- как запускались 1D и 2D расчёты;
- как строились графики и таблицы;
- как оформлялся итоговый отчёт.

7.2 Что должен рассказывать Павел

Павел должен уверенно рассказывать:

- почему выбран такой математический фокус;
- почему сравниваются именно эти аппроксимации;
- почему используются именно эти нормы;
- почему важен радиус локальности;
- как интерпретировать результаты;
- какие выводы можно и нельзя делать.

8 Какие задачи нельзя отдавать полностью Илье

Следующие задачи нельзя отдавать Илье как единственному ответственному:

- выбор и окончательная формулировка аппроксимаций;
- постановка основной гипотезы;
- интерпретация сравнений между моделями;
- объяснение, почему одна аппроксимация лучше другой;
- формулировка математических выводов.

Эти задачи должны оставаться в зоне ответственности Павла.

9 Какие задачи идеально подходят Илье

Следующие задачи как раз подходят Илье:

- собрать каркас проекта;
- запросить у Cursor / GPT skeleton и допилить его;
- наладить окружение;
- запускать обучение;
- прогонять вычисление surface;
- сохранять checkpoints и артефакты;
- строить графики;
- переносить результаты в таблицы;
- собирать всё в `.tex`.

10 Итоговое распределение по deliverables

10.1 За Ильёй закрепить

- структура проекта;
- конфиги и скрипты запуска;
- CIFAR-10 pipeline;
- обучение основной модели;
- обучение второй модели;
- вычисление 1D и 2D landscapes;
- сохранение всех артефактов;
- графики и таблицы;
- сборка и оформление отчёта.

10.2 За Павлом закрепить

- финальную формулировку темы;
- постановку основной и дополнительной гипотезы;
- выбор аппроксимаций;
- математическое описание моделей;
- выбор и смысл норм ошибки;
- анализ влияния радиуса;
- интерпретацию результатов;
- формулировку выводов;
- содержательную часть защиты.

11 Удобное распределение по процентам

Если смотреть не по интеллектуальной сложности, а по объёму руками, то получается такое соотношение:

- **Илья:** около 60–70% общего объёма ручной работы;
- **Павел:** около 30–40% общего объёма ручной работы, но более интеллектуальной и содержательной.

Это является нормальным и честным распределением для проекта такого типа.

12 Распределение по этапам на практике

Этап	Илья	Павел
Структура проекта	создаёт папки, окружение, скрипты	утверждает исследовательскую архитектуру
Обучение моделей	запускает и сохраняет checkpoint'ы	выбирает модели и режимы
1D/2D landscape	прогоняет расчёты и сохраняет результаты	задаёт диапазоны, сетки и обязательные графики
Аппроксимации	считает и оформляет результаты fit	выбирает семейство аппроксимаций и интерпретирует
Нормы ошибок	считает таблицы и графики ошибок	определяет смысл метрик и анализирует выводы
Радиус локальности	запускает серию прогонов	анализирует зависимость и формулирует выводы
Вторая архитектура	повторяет pipeline	решает, что именно проверять
Отчёт	собирает документ и оформление	пишет математическую и аналитическую часть
Защита	техническая реализация и pipeline	математический смысл и результаты

13 Финальный вывод

Проект очень естественно делится на двух участников именно по следующей схеме:

- **Илья** тянет всю инженерно-черновую, запусковую, оформительскую и артефактную часть;
- **Павел** тянет математическое ядро, постановку, анализ аппроксимаций и выводы.

Такое разделение:

- 1) соответствует реальной природе проекта;
- 2) позволяет не терять качество математической части;
- 3) даёт Илье полноценную и полезную роль;
- 4) делает проект устойчивым и управляемым.

Итог

Если кратко зафиксировать роли, то получается так:

Илья — инженер-реализатор, сборщик пайплайна, человек артефактов и оформления.

Павел — исследователь-аналитик, автор математического ядра, интерпретации и выводов.

Именно такое разделение для данного проекта выглядит наиболее продуктивным и честным.